Web scraping with Python

Victor Lee

28-Aug-2020

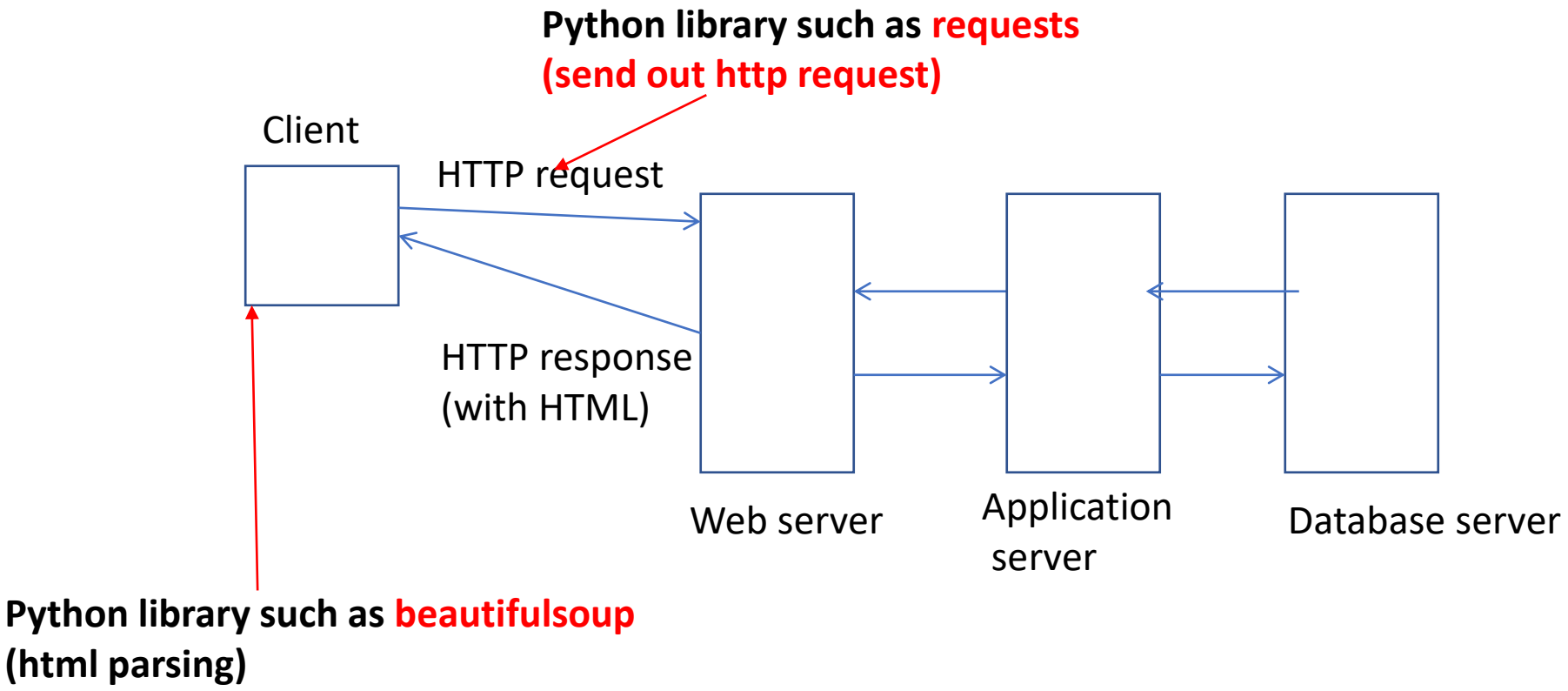
Use of web scraping techniques

- Overview of Web scraping techniques
  - Protocols involved in web browsing
  - Challenges and resolutions involved in web scraping
- Overview of HTML tags
- Use of BeatifulSoup for web scraping
  - HTML parsing tehniques
  - Exercises
- Use of Selenium and BeatifulSoup for web scraping in Python
  - What is Selenium and what we could do with it
  - Exercises

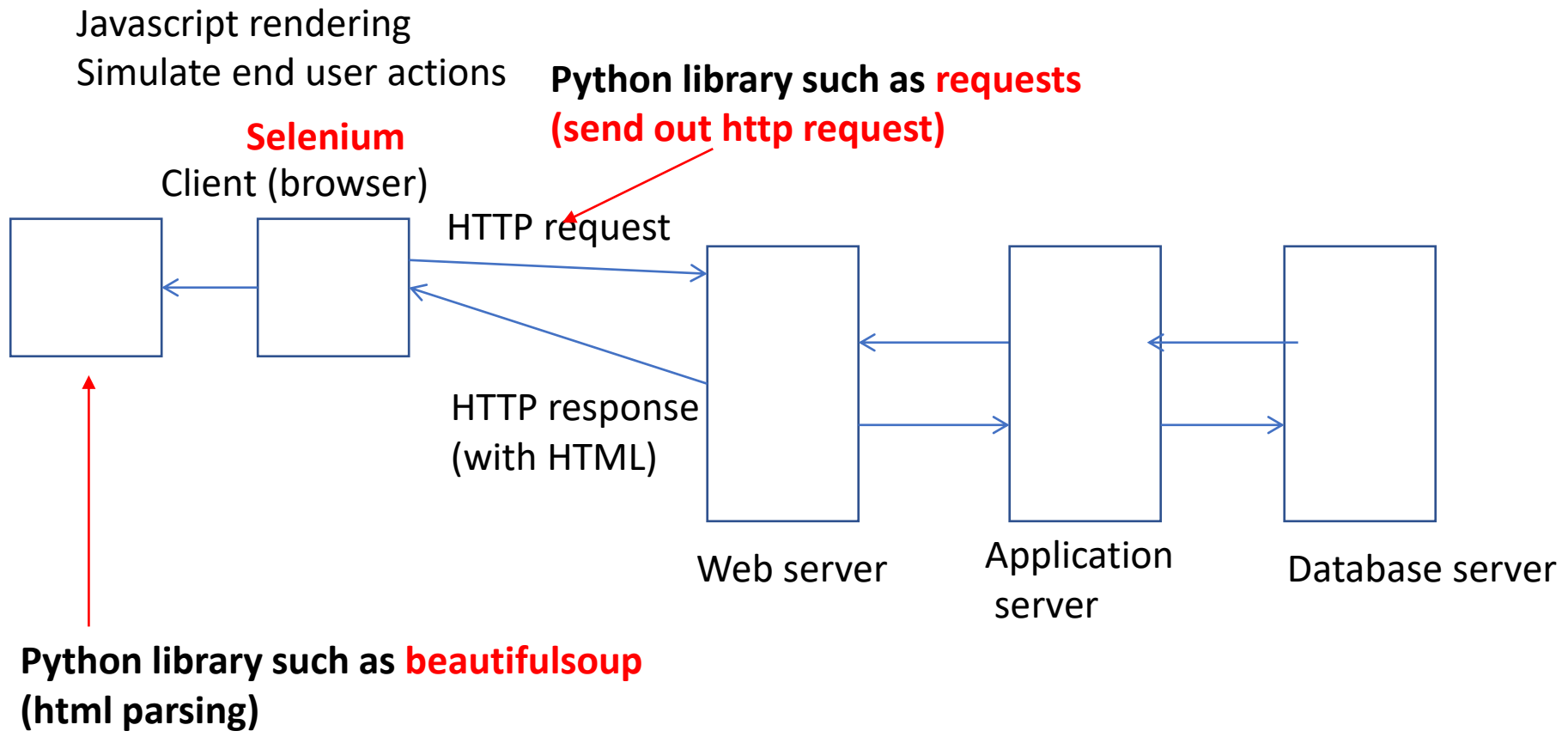
Use of web scraping techniques

- **Overview of Web scraping techniques**
  - **Protocols involved in web browsing**
  - **Challenges and resolutions involved in web scraping**
- Overview of HTML tags
- Use of BeatifulSoup for web scraping
  - HTML parsing tehniques
  - Exercises
- Use of Selenium and BeatifulSoup for web scraping in Python
  - What is Selenium and what we could do with it
  - Exercises

Use of web scraping (with beautifulsoup)



Use of web scraping (Selenium + Beautifulsoup)



Ways to block web scraping (?)

- To identify web scrapers from normal web users
- Traffic rate control
- User authentication
- Capcha
- …

Use of web scraping techniques

- Overview of Web scraping techniques
  - Protocols involved in web browsing
  - Challenges and resolutions involved in web scraping
- **Overview of HTML tags**
- Use of BeatifulSoup for web scraping
  - HTML parsing tehniques
  - Exercises
- Use of Selenium and BeatifulSoup for web scraping in Python
  - What is Selenium and what we could do with it
  - Exercises

What is a Web Page?

- Web pages are text files containing HTML
- HTML – Hyper Text Markup Language
  - A notation for describing
    - document structure (semantic markup)
    - formatting (presentation markup)
  - Looks (looked?) like:
    - A Microsoft Word document
- The markup tags provide information about the page content structure

Creating HTML Pages

- An HTML file must have an `.htm` or `.html` file extension
- HTML files can be created with text editors:
  - NotePad, NotePad ++, PSPad
- Or HTML editors (WYSIWYG Editors):
  - Microsoft FrontPage
  - Macromedia Dreamweaver
  - Netscape Composer
  - Microsoft Word
  - Visual Studio

HTML Structure

- HTML is comprised of “elements” and “tags”
  - Begins with <html> and ends with </html>
- Elements (tags) are nested one inside another:

```
<html> <head></head> <body></body> </html>
```

- Tags have attributes:

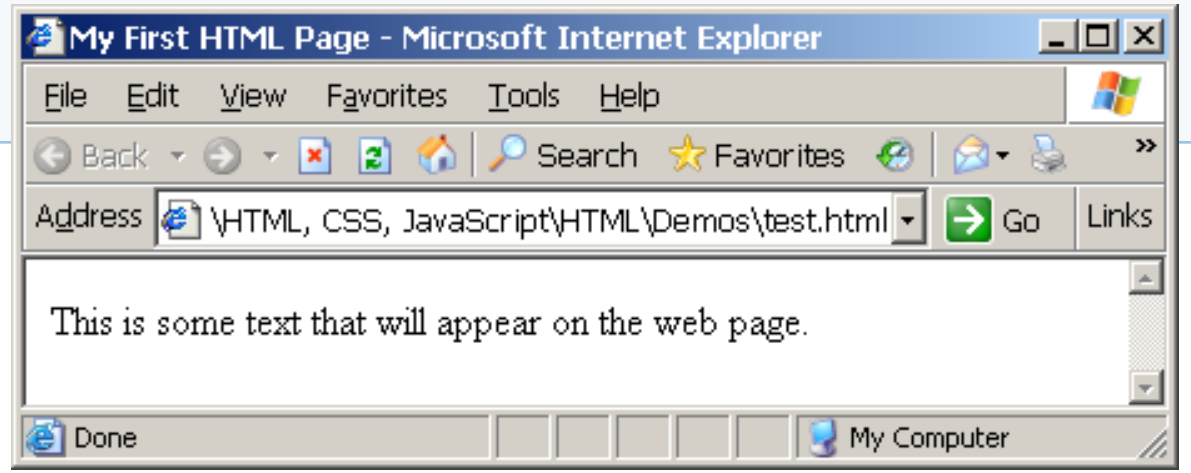
```
<img src="logo.jpg" alt="logo" />
```

- HTML describes structure using two main sections: <head> and <body>

First HTML Page

test.html

```
<!DOCTYPE HTML>
<html>
  <head>
    <title>My First HTML Page</title>
  </head>
  <body>
     <p>This is some text...</p>
  </body>
</html>
```



Some Simple Tags

- Hyperlink Tags

```
<a href="http://www.telerik.com/"
  title="Telerik">Link to Telerik Web site</a>
```

- Image Tags

```
<img src="logo.gif" alt="logo" />
```

- Text formatting tags

```
This text is <em>emphasized.</em>
<br />new line<br />
This one is <strong>more emphasized.</strong>
```

Some Simple Tags – Example

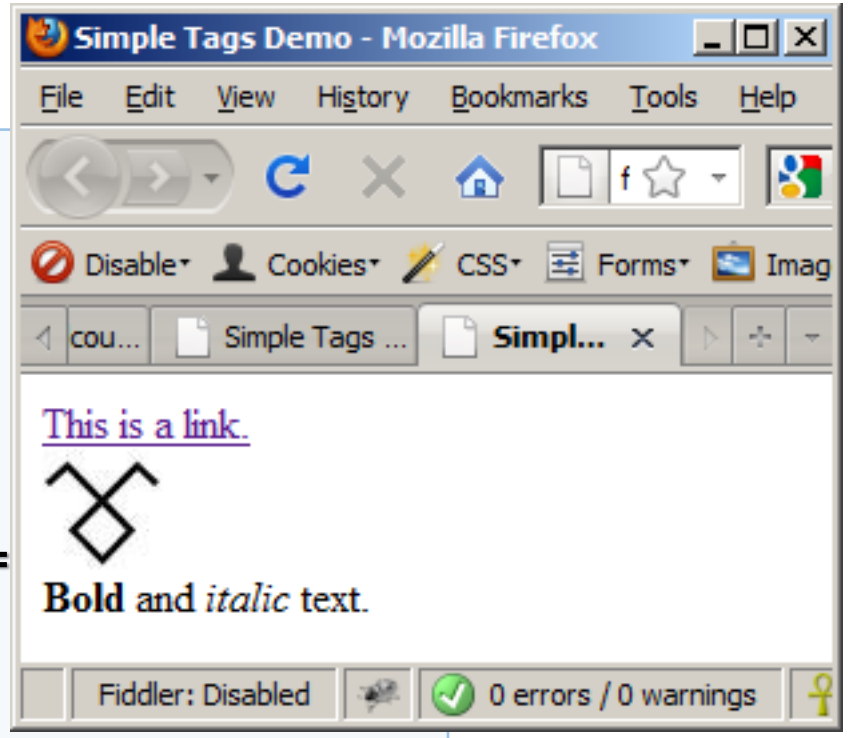
some-tags.html

```
<!DOCTYPE HTML>
<html>
<head>
  <title>Simple Tags Demo</title>
</head>
<body>
<a href="http://www.telerik.com/" title=
    "Telerik site">This is a link.</a>
<br />
<img src="logo.gif" alt="logo" />
<br />
<strong>Bold</strong> and <em>italic</em> text.
</body>
</html>
```

Some Simple Tags – Example (2)

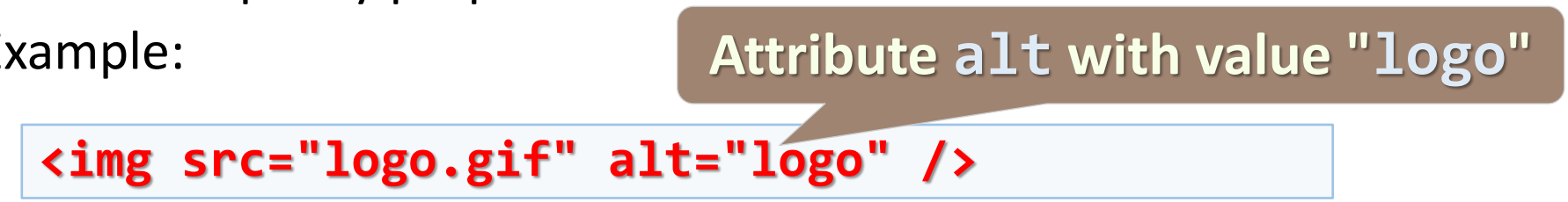
some-tags.html

```
<!DOCTYPE HTML>
<html>
<head>
  <title>Simple Tags Demo</title>
</head>
<body>
<a href="http://www.telerik.com/" title=
    "Telerik site">This is a link.</a>
<br />
<img src="logo.gif" alt="logo" />
<br />
<strong>Bold</strong> and <em>italic</em> text.
</body>
</html>
```



Tags Attributes

- Tags can have attributes
  - Attributes specify properties and behavior
  - Example:



  - Few attributes can apply to every element:
    - `id`, `style`, `class`, `title`
    - The `id` is unique in the document
    - Content of `title` attribute is displayed as hint when the element is hovered with the mouse
    - Some elements have obligatory attributes

Headings and Paragraphs

- Heading Tags (h1 – h6)

```
<h1>Heading 1</h1>
<h2>Sub heading 2</h2>
<h3>Sub heading 3</h3>
```

- Paragraph Tags

```
<p>This is my first paragraph</p>
<p>This is my second paragraph</p>
```

- Sections: `div` and `span`

```
<div style="background: skyblue;">
   This is a div</div>
```

Headings and Paragraphs – Example

headings.html

```
<!DOCTYPE HTML>
<html>
  <head><title>Headings and paragraphs</title></head>
  <body>
    <h1>Heading 1</h1>
    <h2>Sub heading 2</h2>
    <h3>Sub heading 3</h3>

    <p>This is my first paragraph</p>
    <p>This is my second paragraph</p>

    <div style="background:skyblue">
      This is a div</div>
  </body>
</html>
```

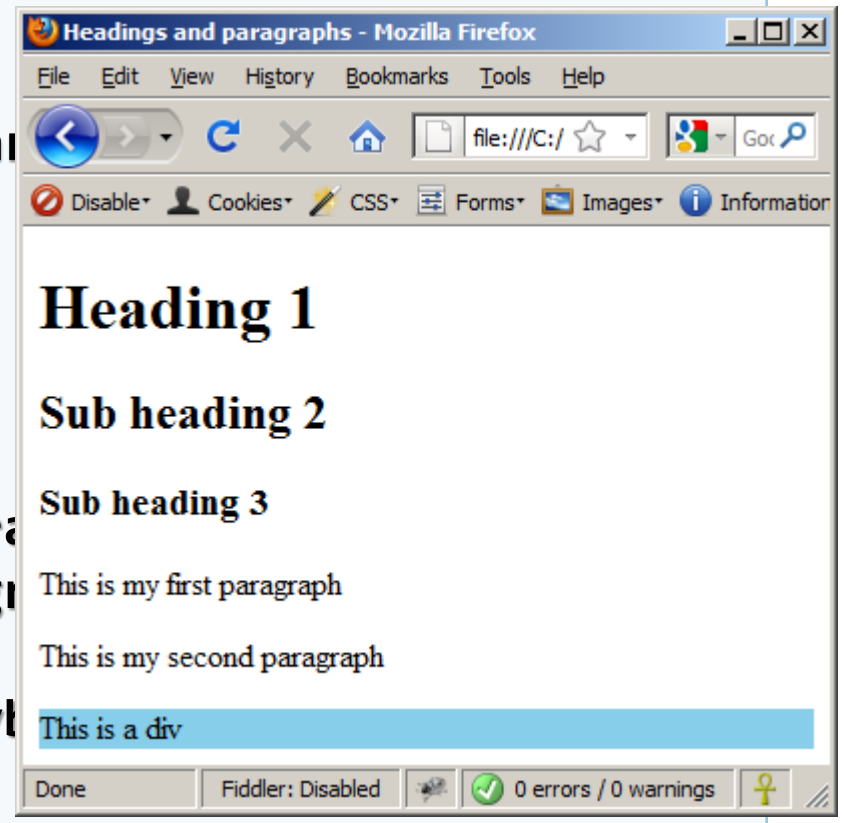
Headings and Paragraphs – Example (2)

headings.html

```
<!DOCTYPE HTML>
<html>
  <head><title>Headings and pa
  <body>
    <h1>Heading 1</h1>
    <h2>Sub heading 2</h2>
    <h3>Sub heading 3</h3>

    <p>This is my first paragra
    <p>This is my second paragr

    <div style="background:skyb
      This is a div</div>
  </body>
</html>
```



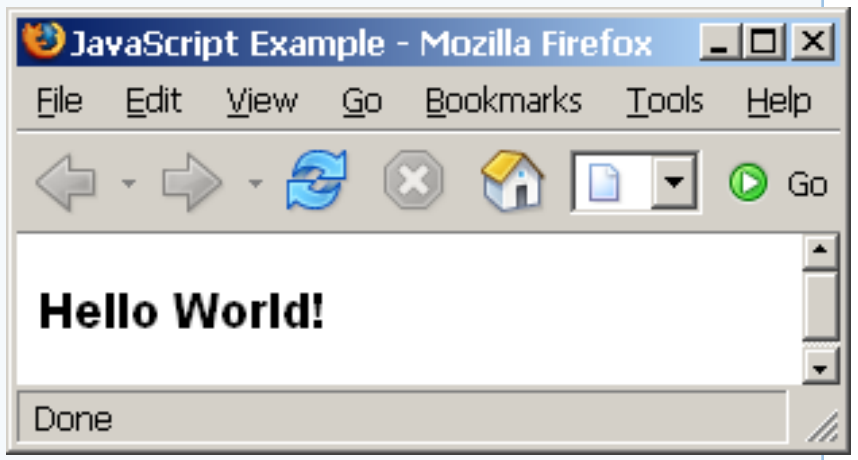
<head> Section: <script>

- The `<script>` element is used to embed scripts into an HTML document
  - Script are executed in the client's Web browser
  - Scripts can live in the `<head>` and in the `<body>` sections
- Supported client-side scripting languages:
  - JavaScript (it is not Java!)
  - VBScript
  - JScript

The <script> Tag – Example

scripts-example.html

```
<!DOCTYPE HTML>
<html>
  <head>
    <title>JavaScript Example</title>
    <script type="text/javascript">
      function sayHello() {
        document.write("<p>Hello World!<\/p>");
      }
    </script>
  </head>
  <body>
    <script type=
      "text/javascript">
      sayHello();
    </script>
  </body>
</html>
```

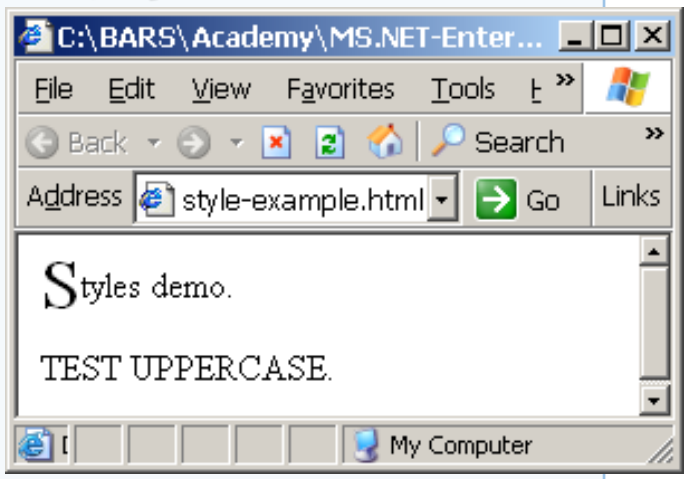


<head> Section: <style>

- The `<style>` element embeds formatting information (CSS styles) into an HTML page

style-example.html

```
<html>
  <head>
    <style type="text/css">
      p { font-size: 12pt; line-height: 12pt; }
      p:first-letter { font-size: 200%; }
      span { text-transform: uppercase; }
    </style>
  </head>
  <body>
    <p>Styles demo.<br />
       <span>Test uppercase</span>.
    </p>
  </body>
</html>
```



Comments: <!-- --> Tag

- Comments can exist anywhere between the `<html></html>` tags
- Comments start with `<!--` and end with `-->`

```
<!-- Telerik Logo (a JPG file) -->
<img src="logo.jpg" alt="Telerik Logo">
<!-- Hyperlink to the web site -->
<a href="http://telerik.com/">Telerik</a>
<!-- Show the news table -->
<table class="newstable">
...
```

<body> Section: Introduction

- The <body> section describes the viewable portion of the page
- Starts after the <head> </head> section
- Begins with <body> and ends with </body>

```
<html>
  <head><title>Test page</title></head>
  <body>
    <!-- This is the Web page body -->
  </body>
</html>
```

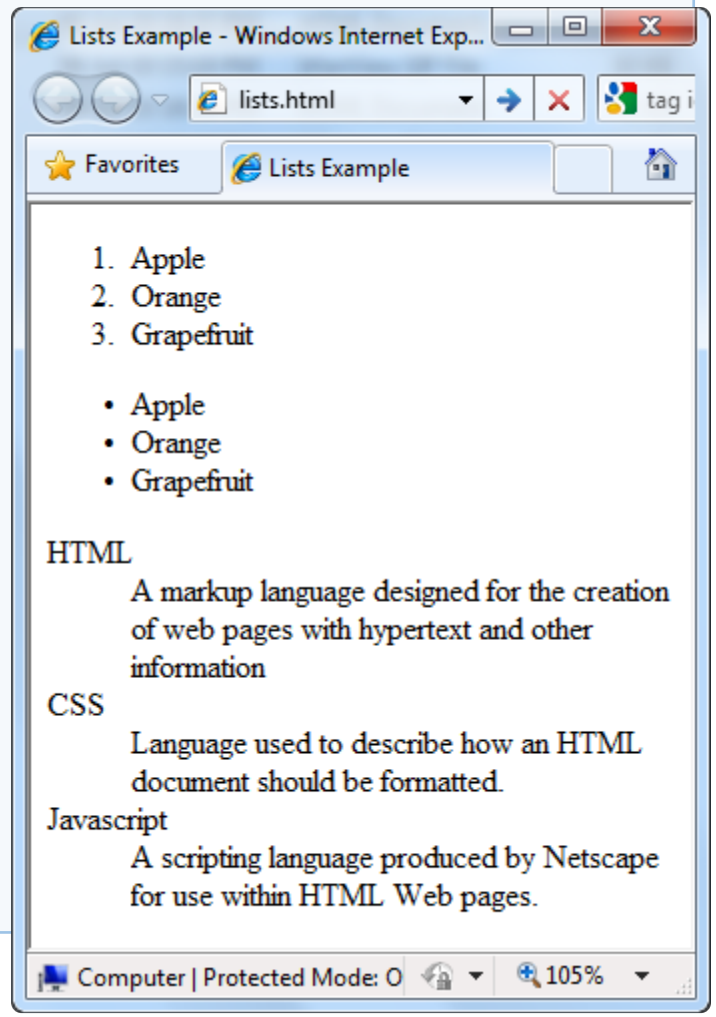
Lists – Example

```
<ol type="1">
    <li>Apple</li>
    <li>Orange</li>
    <li>Grapefruit</li>
</ol>

<ul type="disc">
    <li>Apple</li>
    <li>Orange</li>
    <li>Grapefruit</li>
</ul>

<dl>
    <dt>HTML</dt>
    <dd>A markup lang…</dd>
</dl>
```

lists.html



HTML Tables

- Tables represent tabular data
  - A table consists of one or several rows
  - Each row has one or more columns
- Tables comprised of several core tags: `<table></table>`: begin / end the table

  `<tr></tr>`: create a table row

  `<td></td>`: create tabular data (cell)
- Tables should not be used for layout. Use CSS floats and positioning styles instead

HTML Tables (2)

- Start and end of a table

```
<table> ... </table>
```

- Start and end of a row

```
<tr> ... </tr>
```

- Start and end of a cell in a row

```
<td> ... </td>
```

Simple HTML Tables – Example

```
<table cellspacing="0" cellpadding="5">
  <tr>
    <td><img src="ppt.gif"></td>
    <td><a href="lecture1.ppt">Lecture 1</a></td>
  </tr>
  <tr>
    <td><img src="ppt.gif"></td>
    <td><a href="lecture2.ppt">Lecture 2</a></td>
  </tr>
  <tr>
    <td><img src="zip.gif"></td>
    <td><a href="lecture2-demos.zip">
      Lecture 2 - Demos</a></td>
  </tr>
</table>
```

CSS

……

……

……

……

CSS

Class: Green
Colour="green"

Class: Red
Colour="red"

Use of CSS

- CSS is a language that describes the style of an HTML document.
- CSS describes how HTML elements should be displayed

Selector	Example	Example description
.class	.intro	Selects all elements with class="intro"
#id	#firstname	Selects the element with id="firstname"
*	*	Selects all elements
element	p	Selects all <p> elements
element,element	div, p	Selects all elements and all <p> elements
element element	div p	Selects all <p> elements inside elements
element>element	div > p	Selects all <p> elements where the parent is a element
element+element	div + p	Selects all <p> elements that are placed immediately after elements
element1~element2	p ~ ul	Selects every element that are preceded by a <p> element

CSS selector attribute-Class and ID

- In the CSS, a class selector is a name preceded by a **full stop** (“.”) and an ID selector is a name preceded by a **hash character** (“#”).
- The difference between an ID and a class is that an ID can be used to identify one element, whereas a class can be used to identify more than one.

Use of web scraping techniques

- Overview of Web scraping techniques
  - Protocols involved in web browsing
  - Challenges and resolutions involved in web scraping
- Overview of HTML tags
- **Use of BeatifulSoup for web scraping**
  - **HTML parsing tehniques**
  - **Exercises**
- Use of Selenium and BeatifulSoup for web scraping in Python
  - What is Selenium and what we could do with it
  - Exercises

HTTP request and reply

```
import requests
html=requests.get("http://www.pythonscraping.com/exercises/exercise1.html")
```

- It will send out HTTP get request with the URL link provided.
- The reply will be stored in a request object html
- For the request object we could use the following methods to get more detailed information:
  - request.headers
  - status.code

Example: Modify the user-agent of the http request

```
In [60]: headers={"User-Agent":
          "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/70.0.3538.77 Safari/537.36"}
```

```
In [37]: html=requests.get("http://www.pythonscraping.com/exercises/exercise1.html",headers=headers)
```

```
In [57]: html.request.headers
```

```
Out[57]: {'User-Agent': 'Mozilla/5.0 (Linux; Android 5.1.1; SM-G928X Build/LMY47X) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/47.0.25
         26.83 Mobile Safari/537.36', 'Accept-Encoding': 'gzip, deflate', 'Accept': '*/*', 'Connection': 'keep-alive', 'Cookie': '_pxhd=
         1d875400629f6659dd4ecaa719e6cdbb228ad0a4015b5f8695901b0fa81cba1a:daaae031-ab91-11e9-95ab-a9f9f0284f64'}
```

Use of beatifulsoup as html parser

Type of parser (e.g. lxml, html.parser)

In [39]:
```
from bs4 import BeautifulSoup
```

```
In this example we are going to parse the h1 tags of the web content based on the url
```

In [89]:
```
html=requests.get("http://www.pythonscraping.com/exercises/exercise1.html")
bsObj = BeautifulSoup(html.text,"lxml")
print(bsObj.h1)
```

```
<h1>An Interesting Title</h1>
```

```
You could use get_text() function to get the text of the corresponding html tag
```

In [43]:
```
print(bsObj.h1.get_text())
```

```
An Interesting Title
```

Html parser type

- lxml: faster but require the html content fulfills the standard and specification or else it could not parse correctly
- Html.parser: slower but more robust

Beatifulsoup: findAll/find method

In [68]:

```
bsObj = BeautifulSoup(html, "html.parser")
nameList = bsObj.findAll("span", {"class":"priceText__1853e8a5"})
for name in nameList:
    print(name.get_text())
```

```
6.02
```

It will get the html tags and its content which fulfill the filtering requirement.
In this example it will find all the span tags in which the class value is
equal to priceText_1853e8a5

The result will be stored as python list object

For find method it will get the first tag which fulfill the requirement (from top to down of the html page)

Example: Fetching the table into dataframe

To parse the http table

```
In [75]: import pandas as pd
```

```
In [78]: html = urlopen("http://www.pythonscraping.com/pages/page3.html")
         bsObj = BeautifulSoup(html, "lxml")
         table=bsObj.find("table",{"id":"giftList"})
         df=pd.read_html(str(table),header=0)
         df[0]
```

Header=0 => the first row of fetched data is the header field
and it will become the columns of the dataframe
The result will be returned as python list object and hence
You need to get the first list element

Use of web scraping techniques

- Overview of Web scraping techniques
  - Protocols involved in web browsing
  - Challenges and resolutions involved in web scraping
- Overview of HTML tags
- Use of BeatifulSoup for web scraping
  - HTML parsing tehniques
  - Exercises
- **Use of Selenium and BeatifulSoup for web scraping in Python**
  - **What is Selenium and what we could do with it**
  - **Exercises**

Use of Selenium

- So far beatifulsoup is good enough to be an effective html parser
- However in some cases we may need to perform the following tasks:
  - To render the javascript content (javascript agent) and get the rendered content for further html parser
  - To automate end user actions (click the button/sending text in the input box/user login etc)
- In such case Selenium will be used to achieve the tasks

Example of using Selenium

Trigger the chrome driver and launch a chrome browser session. The session will be assigned By the variable (e.g. driver) and then you could Manipulate the browser activity with the variable

```
In [100]: from selenium import webdriver
          import time
          driver = webdriver.Chrome('E:/R_Working_Directory/chromedriver.exe')
          driver.get('http://www.hkex.com.hk/Mutual-Market/Stock-Connect?sc_lang=en')
```

```
In [101]: html=driver.page_source
```

Get the rendered page content with the attribute page_source

```
In [102]: soup = BeautifulSoup(html, "lxml")
          sz = soup.find("div", {"class":"rightpane itemtrad"})
          szsb = sz.find("div", {"class":"row southbound"})
          szsbval = szsb.find("div", {"class":"type value"})
          val_str = szsbval.get_text()
          val = int(val_str.replace("HK$", "").replace(",", "").replace("M", ""))*1000000
          val

Out[102]: 2538000000
```

Use of WebDriverWait function

```
In [103]: from selenium import webdriver
          from selenium.webdriver.common.by import By
          from selenium.webdriver.support.ui import WebDriverWait
          from selenium.webdriver.support import expected_conditions as EC

          driver = webdriver.Chrome("E:/R_Working_Directory/chromedriver.exe")
          driver.get('https://www.hkex.com.hk/Market-Data/Futures-and-Options-Prices/Equity-Index/Mini-Hang-Seng-Index-Futures-and-Options

          try:
              element = WebDriverWait(driver, 30).until(
                  EC.element_to_be_clickable((By.CLASS_NAME, "ns_night")))
          except:
              driver.quit()

          element.click()
```

With AJAX technology some of the html
Content may not be available after the html
Reply was retrieved.
Hence in such case we could better use
This way to wait till the requested html tag and
Its content is available.

It will wait for 30 seconds
Before the requested item
Is clickable or else it will then
Quit the browser session

https://selenium-python.readthedocs.io/waits.html